

AP9EV – Genetický algoritmus, binární problém

František Hromek

Model: Claude Opus 5

Dělám do školy genetický algoritmus s binární reprezentací jedinců. Než začneem řešit kód, vysvětli mi prosím jak GA funguje ve zkratce krok za krokem, pipeline by měla být něco jako: populace - selekce - křížení - mutace - nová populace.

- Tady provedl čistě základní vysvětlení, snad není třeba, byla to hromada textu, vysvětlil to hezky

V tomto repu každá složka bude reprezentovat řešení jednoho úkolu, takže v rámci GeneticAlgorithm založ nějaký ga.py nebo main, nechám na tobě a nachystej kod pro vytvoření základní populace (seznam binárních řetězců, hádám, že listy intů 0/1. Napiš funkci, která vygeneruje náhodnou populaci o velikosti pop_size a délce jedince D. At to jede přes standartní knihovny + numpy

```
from __future__ import annotations

import numpy as np

# Populace: np.ndarray tvaru (pop_size, D), dtype uint8, hodnoty pouze 0/1.
Population = np.ndarray

def random_population(
    pop_size: int,
    D: int,
    rng: np.random.Generator | None = None,
) -> Population:
    """Vygeneruje náhodnou počáteční populaci.

    Každý bit je nezávisle 0 nebo 1 s pravděpodobností 0.5, takže je
    populace rovnoměrně rozprostřená po celém prohledávaném prostoru.

    Args:
        pop_size: Počet jedinců v populaci.
        D: Délka chromozomu (počet bitů jednoho jedince).
        rng: Generátor náhodných čísel. Předdej vlastní se seedem, pokud
            chceš reprodukovatelný běh; jinak se použije nedeterministický.

    Returns:
        Matice tvaru (pop_size, D) s hodnotami 0/1.

    Raises:
```

```

        ValueError: Pokud pop_size nebo D není kladné.
    """
    if pop_size <= 0:
        raise ValueError(f"pop_size musí být kladné, dostal jsem {pop_size}")
    if D <= 0:
        raise ValueError(f"D musí být kladné, dostal jsem {D}")

    rng = np.random.default_rng() if rng is None else rng
    return rng.integers(0, 2, size=(pop_size, D), dtype=np.uint8)

def chromosome_to_str(chromosome: np.ndarray) -> str:
    """Převéde chromozom na řetězec bitů kvůli čitelnému výpisu."""
    return "".join(str(bit) for bit in chromosome)

def main() -> None:
    pop_size, D = 10, 16

    rng = np.random.default_rng(seed=42)
    population = random_population(pop_size, D, rng)

    print(f"Populace: {population.shape[0]} jedinců, délka chromozomu {population.shape[1]}")
    for i, individual in enumerate(population):
        print(f" {i:2d}: {chromosome_to_str(individual)} (jedniček: {individual.sum()})")

    print(f"\nPodíl jedniček v populaci: {population.mean():.3f} (očekáváno ~0.5)")

if __name__ == "__main__":
    main()

```

Ted' potřebuji dvě účelovky, OneMax a LeadingOnes pokud jsem to správně pochopil, klidně mi o něco velmi nich krátce něco vysvětlí.

Ano, pochopil jsi to správně. Obě jsou v GeneticAlgorithm/ga.py, ověřeno včetně hraničních případů (1D i 2D vstup, samé nuly, samé jedničky, D=300 bez přetečení).

OneMax

Fitness = počet jedniček. 10110 → 3. Optimum je D.

Každý bit přispívá +1 nezávisle na všech ostatních — funkce je plně separabilní a unimodální, žádná lokální optima. Proto je to sanity check: když ti GA nezvládne OneMax, je chyba v implementaci, ne v obtížnosti úlohy. Zvládne ji i triviální horolezec.

LeadingOnes

Fitness = délka úvodního bloku jedniček, než přijde první nula. 1110100 → 3. Optimum je taky D.

Tady je vtip v tom, že bit na pozici i se počítá jen když jsou všechny bity před ním jedničky. Zlepšit jedince znamená najít tu jednu konkrétní nulu na prvním neúspěšném místě — všechny ostatní bity jsou v daný moment „neviditelné“. To je epistáze: hodnota genu závisí na kontextu ostatních genů.

Praktické důsledky, které uvidíš v grafech:

- Konverguje pomaleji. OneMax potřebuje $\sim D \cdot \ln(D)$ vyhodnocení, LeadingOnes $\sim D^2$. Při $D = 100$ je to znatelný rozdíl.

- Průběh je schodovitý, ne hladký. Fitness dlouho stojí a pak skočí o několik bodů naráz — když se překlopí blokující nula, „odemkne“ se rovnou několik jedniček, které za ní už náhodou byly.

- Křížení tu funguje jinak. Jednobodové křížení dobrý prefix buď zachová, nebo ho utne — pozice řezu má smysl. U OneMaxu je poloha řezu jedno, záleží jen na počtu jedniček. Právě proto se ta dvojice zadává společně: vedle sebe ukážou, jestli tvůj operátor umí něco víc než jen sčítat bity.

Všimni si toho i na ukázkovém výpisu výše: náhodná populace má v OneMaxu průměr 8.5 z 16 (přesně polovina, jak se čeká), ale v LeadingOnes průměr 0.40 — víc než polovina jedinců má nulu hned na první pozici, takže fitness 0. LeadingOnes dává na startu skoro nulový gradient, GA se musí „chytit“ nejdřív něčeho.

K implementaci

```
def one_max(population):
```

```
    return population.sum(axis=-1, dtype=np.int64)
```

```
def leading_ones(population):
```

```
    return np.cumprod(population, axis=-1, dtype=np.int64).sum(axis=-1)
```

Dvě drobnosti, které stojí za pozornost:

- axis=-1 znamená „poslední osa“, takže obě funkce berou bez úprav jak jednoho jedince (1D), tak celou populaci (2D) — pro populaci vrátí rovnou vektor fitness, žádný cyklus.

- Trik u LeadingOnes: kumulativní součin je 1, dokud jdou samé jedničky, a od první nuly už zůstane 0 navždy. Jeho součet je přesně délka úvodního bloku. dtype=np.int64 je tam proto, že populace je uint8 a součet nad 255 bity by přetekl.

Obě maximalizuješ a obě mají optimum D — hodí se to jako jednotná ukončovací podmínka (best == D) i na porovnání průběhů v jednom grafu.

Další krok je selekce. Turnaj?

Napiš funkci pro jednobodové křížení dvou rodičů: náhodně se zvolí bod, řetězce se v něm rozdělí a první části se prohodí a vzniknou 2 potomci. Přidej mi k tomu pár vět, jak to funguje, ať to pobírám

rodič 1: 1011 | 0100 potomek 1: 1101 0100

rodič 2: 1101 | 0011 → potomek 2: 1011 0011

↑

bod řezu = 4

Vylosuje se jedno číslo point, oba chromozomy se v něm rozseknou a první části se prohodí. Potomek 1 dostane začátek od rodiče 2 a konec od rodiče 1, potomek 2 obráceně.

Klíčová vlastnost: nevzniká tu žádná nová informace. Oba potomci dohromady obsahují přesně tytéž bity jako oba rodiče, jen přeskádané. Když mají oba rodiče na dvacáté pozici nulu, budou ji mít i oba potomci — křížení sáhne jen po tom, co už v populaci je. Nové bity umí přinést výhradně mutace. Proto se křížení označuje jako exploatace (využívá stávající materiál) a mutace jako explorace.

Smysl operace je, že když jeden rodič má dobrý začátek a druhý dobrý konec, křížení je může spojit do jednoho lepšího potomka. Riziko je zrcadlové — může stejně dobře rozbít fungující kombinaci. Proto se používá pravděpodobnost p_c (typicky 0.8–0.9): část párů se zkříží, část projde do nové generace beze změny.

+ dodal kód

Přidej jednobodovou mutaci: projdi potomka bit po bitu a s pravděpodobností p_{mut} (řádově 0.5–1 %) invertuj daný bit.

```
def mutate(
    individual: np.ndarray,
    p_mut: float,
    rng: np.random.Generator | None = None,
) -> np.ndarray:
    """Bitová mutace: každý bit se nezávisle invertuje s pravděpodobností
    p_mut.
```

Mutace je jediný operátor, který umí do populace vnést bit, jenž v ní dosud nebyl — drží tedy diverzitu a je pojistkou proti uváznutí v lokálním optimu. Rozumná volba je p_{mut} kolem $1/D$, tedy v průměru jeden překlopený bit na jedince.

Args:

*individual: Chromozom (1D) nebo celá populace (2D) k mutaci.
p_mut: Pravděpodobnost překlopení jednoho bitu, typicky 0.005–0.01.
rng: Generátor náhodných čísel; bez něj se použije nedeterministický.*

```

Returns:
    Nové pole stejného tvaru – vstup zůstává nezměněn.

Raises:
    ValueError: Pokud p_mut není z intervalu <0, 1>.
    """
    if not 0.0 <= p_mut <= 1.0:
        raise ValueError(f"p_mut musí být z intervalu <0, 1>, dostal jsem {p_mut}")

    rng = np.random.default_rng() if rng is None else rng

    # Maska říká, které bity se překlápí; XOR s 1 invertuje, XOR s 0 nechá být.
    flips = (rng.random(individual.shape) < p_mut).astype(individual.dtype)
    return individual ^ flips

```

Ted potřebuji selekci rodičům udělejme obě varianty najednou, roulette i rank selection, udělej to tak, at mezi nimi můžu později přepínat.

Přepínání

```
parents = select_parents(population, fitness, method="roulette", rng=rng)
```

```
parents = select_parents(population, fitness, method="rank", rng=rng)
```

Dispatcher je postavený nad registrem:

```
SELECTION_METHODS = {"roulette": roulette_selection, "rank": rank_selection}
```

Obě funkce mají identickou signaturu (population, fitness, n_parents=None, rng=None) -> (n_parents, D), takže je můžeš volat i napřímo, nebo je prohazovat v cyklu — for method in SELECTION_METHODS: ti

projede obě, což se hodí na srovnávací graf do protokolu. Až budeš přidávat turnaj, stačí funkce se stejnou signaturou plus jeden řádek do slovníku, nikde jinde nic neměníš.

Vybírá se s opakováním — to je záměr, ne chyba. Právě tím, že silný jedinec padne víckrát, se jeho geny šíří populací.

Roulette

Pravděpodobnost úměrná fitness: $p_i = f_i / \Sigma f$. Dvojnásobná fitness = dvojnásobná šance.

Dvě místa, kde se to láme, a jak to kód řeší:

- Záporná fitness → ValueError s odkazem na rank. Podíl by dal nesmysl, tiché selhání by ses hledal dlouho.

- Všichni mají 0 → spadne to na rovnoměrný výběr místo dělení nulou. U tebe to není teoretická situace: LeadingOnes má na startu běžně celou populaci na nule (v demu 6 z 10 jedinců), takže bez tohoto ošetření by ti první generace shodila celý běh.

Slabina zůstává i tak: ruleta je citlivá na rozsah fitness. Když má jeden jedinec 100 a zbytek 10, sebere skoro všechny losy a populace zkolabuje (předčasná konvergence). Naopak u fitness 1000 vs. 1001 jsou šance prakticky stejné a selekční tlak zmizí.

Rank

Jedinci se seřadí a dostanou pořadí 1 (nejhorší) až N (nejlepší), pravděpodobnost je $\text{rank} / \Sigma \text{rank}$. Absolutní hodnoty fitness se zahodí, pracuje se jen s pořadím.

Tím zmizí obě slabiny rulety naráz: superjedinec dostane rank N, ne stonásobek, a při fitness 1000 vs. 1001 je pořád rozdíl jednoho ranku. Selekční tlak je konstantní po celý běh, nezávisle na tom, jak

se mění měřítko fitness. Cena je ztráta informace — „o kolik“ je někdo lepší, se nepočítá. A zvládne zápornou fitness, protože se nedělí.

Co je vidět na výstupu

Z výpisu v `main()` (1000 losů, populace seřazená od nejhoršího):

┌──────────┴──────────┴──────────┐		
└ fitness ─┘	└ roulette ─┘	└ rank ─┘
┌──────────┴──────────┴──────────┐		
└ 5 ─┘	└ 55 ─┘	└ 19 ─┘
┌──────────┴──────────┴──────────┐		
└ 6 ─┘	└ 73 ─┘	└ 37 ─┘
┌──────────┴──────────┴──────────┐		
└ 9 ─┘	└ 102 ─┘	└ 81 ─┘
┌──────────┴──────────┴──────────┐		
└ 12 ─┘	└ 135 ─┘	└ 198 ─┘
┌──────────┴──────────┴──────────┐		

Teď to spojíme. Napiš hlavní funkci `run_ga(...)` s nastavitelnými parametry (`pop_size`, `elite_ratio`, `p_mut`, typ selekce, účelová funkce): z populace udělá novou tak, že nejdřív přemístí zlomek nejlepších (elitismus, typicky 10–20 %), a pak dokud není nová populace plná, vybírá 2 rodiče selekcí, udělá jednobodové křížení a zmutuje oba potomky. Důležité: rozpočet neměř v generacích, ale v počtu volání fitness funkce — zadání chce 100×D ohodnocení (1000/3000/10000 pro 10/30/100D), takže běh skončí po vyčerpání rozpočtu. Pak mi tělo `run_ga` projedí řádek po řádku komentářem, ať tomu rozumím, a spusť jeden testovací běh na OneMax v 10D, ať vidím, že to konverguje

`run_ga(...)` je v `GeneticAlgorithm/ga.py:339`. Testovací běh dole, nejdřív průchod kódem.

Signatura

```
run_ga(fitness_fn, D, pop_size=50, elite_ratio=0.1, p_mut=None,  
      p_cross=1.0, selection="roulette", max_evals=None, rng=None)
```

Vrací dataclass `GAResult` s poli `best`, `best_fitness`, `evaluations`, `generations`, `history`.

+ samozřejmě dodal kód

Udělej funkci, co pro danou úlohu a dimenzi spustí 10 nezávislých běhů, během každého ukládá nejlepší fitness podle počtu ohodnocení a z toho spočítá průměrnou konvergenční křivku. Tu vykreslí přes matplotlib (X = počet ohodnocení, Y = průměrná nejlepší fitness). K tomu z finálních výsledků 10 běhů spočítej statistiky: nejlepší, nejhorší, průměr, medián a směrodatnou odchylku a vypiš je do tabulky.

Hotovo — nový soubor `GeneticAlgorithm/experiment.py` (běhy, statistiky, graf) plus drobná úprava `ga.py`: `GAResult` má teď navíc `curve`.

Jak se sbírá křivka

Klíčová věc pro průměrování: `run_ga` nově zaznamenává `curve[i] = nejlepší fitness po i+1 vyhodnoceních`, tedy s granularitou jednoho ohodnocení, ne jedné generace. Díky tomu mají všechny běhy křivku stejné délky $100 \cdot D$ a průměr je prostý `curves.mean(axis=0)` — žádná interpolace na společnou osu. Kdybych to nechal po generacích, každý běh by měl body jinde (počet generací kolísá podle toho, kolik potomků se vejde do rozpočtu) a průměr by se musel dopočítávat.

Prvních `pop_size` hodnot je `np.maximum.accumulate(fitness)` počáteční populace — ta se ohodnotí jedním vektorovým voláním, ale z hlediska rozpočtu to je `pop_size` vyhodnocení, takže křivka musí začít běžícím maximem přes ně.

Ověřeno: křivka je neklesající, má přesně délku rozpočtu a končí na `best_fitness`.

Nezávislost běhů

```
seeds = np.random.SeedSequence(seed).spawn(n_runs)
```

Místo `seed`, `seed+1`, `seed+2...` — sousední seedy mohou dát korelované proudy, `spawn()` garantuje nezávislost. Zároveň je to plně reprodukovatelné, takže tabulka i graf vyjdou pokaždé stejně.

Ověřeno: křivka je neklesající, má přesně délku rozpočtu a končí na `best_fitness`.

Nezávislost běhů

```
seeds = np.random.SeedSequence(seed).spawn(n_runs)
```

Místo *seed*, *seed+1*, *seed+2*... — sousední *seedy* můžou dát korelované proudy, *spawn()* garantuje nezávislost. Zároveň je to plně reprodukovatelné, takže tabulka i graf vyjdou pokaždé stejně.

Je tam *zadani.txt* v tomto adresáři, ověř po experimentech jak jsme na tom vůči zadání. Zabal to do jedné funkce *experiment()*, která projede obě úlohy (*OneMax* i *LeadingOnes*) a všechny tři dimenze (*10/30/100D*), vygeneruje grafy i statistiky, a napiš mi, jak ji spustit

Co změnilo výsledky

*Zadání chce *p_mut* v rozsahu 0.5–1 %, já měl default 1/D — což je při 10D celých 10 %, mimo zadaný rozsah. Při opravě na 0.01 jsem rovnou proladil i zbytek (to zadání taky chce, poslední věta) a defaulty jsou teď:*

DEFAULT_PARAMS = {"pop_size": 20, "elite_ratio": 0.2, "p_mut": 0.01, "p_cross": 1.0}

*Nejdůležitější zjištění z ladění: menší populace je při pevném rozpočtu výrazně lepší. Na *LeadingOnes* 100D, selekce rank, průměr z 10 běhů:*

┌──────────┴──────────┴──────────┐			
pop_size	elitismus 10 %	elitismus 20 %	
└──────────┬──────────┬──────────┘			
20	78.9	82.0	
└──────────┬──────────┬──────────┘			
30	75.6	80.7	
└──────────┬──────────┬──────────┘			
50	66.6	68.1	
└──────────┬──────────┬──────────┘			
100	51.7	54.2	
└──────────┬──────────┬──────────┘			

*Dává to smysl: rozpočet 10 000 ohodnocení je buď 500 generací po 20 jedincích, nebo 100 generací po 100. *LeadingOnes* potřebuje hlavně hodně generací za sebou, protože se prokousává bit po bitu zleva — široká populace mu nepomůže, jen prožere rozpočet.*

*Nové výsledky proti minulým (*pop_size* 50, *p_mut* = 1/D):*

úloha	dřív	teď
OneMax 100D roulette	99.90	100.00 (10/10 běhů)
LeadingOnes 100D roulette	57.20	82.30
LeadingOnes 100D rank	66.60	82.00

LeadingOnes ve 100D se pořád nedojede na optimum (nejlepší jednotlivý běh 99/100), ale z ~60 na ~82 je to velký posun a křivka v pravém panelu je na konci rozpočtu stále rostoucí — to je ten závěr, který má zadání ukázat.

Rozdíl mezi selekcemi se po doladění skoro smazal (82.3 vs 82.0). Při pop_size 50 byl rank jasně lepší, při 20 už na tom nezáleží — malá populace sama o sobě vytváří dost selekčního tlaku. Do protokolu je to pěkná poznámka: parametry spolu interagují, nelze je ladit po jednom.

Zadání chce najít vhodné nastavení parametrů. Zkus proti sobě 2–3 konkrétní nastavení (např. malá vs. větší populace, roulette vs. rank, mutace 0.5 % vs. 1 %), ukaž v grafu, které konverguje líp a shrň mi na pár vět jaké finální nastavení bys doporučil a proč, to použiju jako závěr do protokolu.

Závěr

Doporučené nastavení: velikost populace 20, elitismus 20 %, pravděpodobnost mutace 1 %, pořadová (rank) selekce, křížení vždy.

Nejsilnějším faktorem se ukázala velikost populace. Při pevném rozpočtu $100 \times D$ ohodnocení je populace 20 jedinců výrazně lepší než 50 (LeadingOnes 100D: 85.7 vs. 66.8; OneMax 100D dosáhne optima po 1738 vs. 2863 ohodnoceních). Důvod je v tom, že rozpočet se dělí mezi velikost populace a počet generací — 10 000 ohodnocení je buď 500 generací po 20 jedincích, nebo 200 po 50. LeadingOnes se prokousává řetězcem bit po bitu zleva a potřebuje především hodně generací za sebou; široká populace mu paralelní prohledávání nevykompenzuje.

Snížení mutace na 0.5 % uškodilo (LeadingOnes 30D: 25.4 vs. 29.6), protože při krátkém chromozomu už mutace netrefí blokující nulu dost často a algoritmus stagnuje. Horní hranice rozsahu ze zadání je tedy lepší volba než dolní.

Rank vs. ruleta je nejslabší z testovaných faktorů. Na finálním výsledku LeadingOnes je rozdíl malý (85.7 vs. 81.6), zřetelný je až v rychlosti na OneMaxu (1738 vs. 3659 ohodnocení, tedy víc než dvojnásobek).

Ruleta doplácí na to, že fitness OneMaxu se drží v úzkém pásmu kolem $D/2$, takže poměr $f_i/\Sigma f$ mezi dobrým a špatným jedincem je téměř stejný a selekční tlak prakticky mizí; rank pracuje pořadím, a je proto na měřítku fitness nezávislý.

Za zmínku stojí, že se vliv selekce při malé populaci zmenšil oproti dřívějším běhům s populací 50, kde byl rank jasně lepší. Malá populace sama vytváří dostatečný selekční tlak, takže na volbě selekce už tolik nezáleží. Kontrolní parametry tedy nelze ladit nezávisle na sobě — interagují.

Ani s nejlepším nastavením se LeadingOnes ve 100D nepodařilo spolehlivě vyřešit (průměr 85.7 ze 100, optima dosáhly jen některé běhy). Konvergenční křivka je v okamžiku vyčerpání rozpočtu stále rostoucí, takže limitem není algoritmus, ale rozpočet LeadingOnes vyžaduje řádově D^2 ohodnocení, což je pro $D = 100$ právě na hranici zadaných 10 000.